

MODULE 3 – SHORT NOTES

Web & Application Security

Information Security • B.Tech Computer Engineering • Somaiya Vidyavihar
Course Code: 116U01C602 | Semester VI (SVU 2020)

A. Web Application Vulnerabilities

A web application vulnerability is a security flaw, weakness or misconfiguration that attackers exploit to gain unauthorised access, steal data or disrupt services. They usually stem from poor input validation.

SQL Injection (SQLi)

Attackers insert malicious SQL code into user-input fields, manipulating or deleting backend database content, or escalating privileges.

Types of SQL Injection

Classic SQLi	Directly injects bad commands into a query (most common type).
Union-Based	Uses UNION to extract data from other tables.
Error-Based	Reads database structure from error messages.
Blind SQLi	No error messages shown; attacker guesses via true/false responses or time delays.

Classic Example – Login Bypass

Vulnerable query: `SELECT * FROM USERS WHERE USERNAME='$username' AND PASSWORD='$password'`
Attacker enters password: `' OR '1'='1` → condition is ALWAYS TRUE → login bypassed!

Safe fix – Prepared Statements: `$stmt = $conn->prepare('SELECT * FROM USERS WHERE USERNAME=? AND PASSWORD=?');`

Error-Based Example

Normal query: `SELECT NAME, SALARY FROM CUSTOMERS WHERE ID='1'`

Attack input: `1 OR 1=1` → returns ALL rows, exposing all customer data.

Union-Based Example

`SELECT name, price FROM products UNION SELECT username, password FROM users;` — fetches credentials by appending a second query.

Prevention of SQL Injection

- Use Prepared Statements / Parameterised Queries — treats input as data, not code.
- Use Stored Procedures — SQL logic lives inside the DB.
- Validate & sanitise all input.
- Apply least-privilege to DB accounts.
- Use ORM libraries (Hibernate, Entity Framework).
- Disable detailed error messages.

 **Key trick:** Prepared statements separate code from data, making injection impossible.

 **PYQ: 2023 Q3 – Discuss methodologies used for web application hacking. | 2024 Q3 – Email-based attacks and countermeasures.**

▶ Cross-Site Scripting (XSS)

XSS lets an attacker inject malicious JavaScript into web pages viewed by innocent users. It bypasses the Same-Origin Policy, allowing session hijacking, data theft and full account compromise.

◆ Types of XSS

Reflected XSS	Malicious script comes from the current HTTP request (e.g., in a URL parameter). Simplest form.
Stored XSS	Malicious script is stored in the database and served to all users. Also called Persistent/Second-Order XSS.
DOM-Based XSS	Vulnerability is in client-side JavaScript that writes untrusted data directly into the DOM.

◆ Reflected XSS Example

URL: `https://site.com/status?message=<script>/*evil*/</script>`

→ Server echoes the script into the HTML; victim's browser executes it.

◆ Stored XSS Example

Attacker posts: `<p><script>/*evil*/</script></p>` in a message board.

→ Every user who views the board runs the attacker's script.

◆ DOM XSS Example

JS: `results.innerHTML = 'You searched for: ' + search;`

→ Attacker inputs: `<img src=1 onerror='evil()'>` and the script executes.

💡 Remember: Reflected = URL-based, Stored = database, DOM = client-side JS.

🔗 PYQ: 2023 Q3i – Website hacked, homepage replaced. What happened and how to avoid? (This is a Stored XSS / Defacement scenario.)

▶ Cross-Site Request Forgery (CSRF)

CSRF tricks an authenticated user's browser into sending an unwanted request to a different website. It exploits the trust the site has in the user's browser. The user doesn't know the action was performed.

- Victim must be logged in to the target site.
- Attacker crafts a forged request (e.g., hidden form, image tag).
- Prevention: CSRF tokens, SameSite cookies, referrer validation.

💡 Difference: XSS exploits the user's trust in the website; CSRF exploits the website's trust in the user's browser.

B. Threat Modelling

Threat modelling is a structured process to identify, analyse and mitigate potential threats before attackers exploit them. Security is built into the design phase.

- Identifies what can go wrong before deployment.
- Understands attacker goals, entry points and techniques.
- Prioritises risks based on impact and likelihood.

◆ Can be applied to:

Software & applications, Systems & networks, Distributed systems & IoT, Business processes.

▶ Process of Threat Modelling (6 Steps)

1. Define Scope & Objectives	Identify what needs protection, define assets, boundaries, goals, and who makes security decisions.
---	---

2. Diagram the System	Create a visual map of components and data flows. Highlights trust boundaries and attack areas.
3. Identify Threats	Determine what can go wrong. Uses STRIDE or similar frameworks. Maps threats to assets.
4. Analyse & Prioritise Risks	Rank threats by impact and likelihood. Estimate real-world damage potential.
5. Design Mitigations	Plan security controls to reduce or eliminate threats. Aligns controls to specific threats.
6. Review & Iterate	Continuously refine as the system evolves. Validate controls, update diagrams.

Threat Modelling Methodologies

◆ STRIDE (Microsoft) — Most Important!

S – Spoofing	Pretending to be someone/something else.
T – Tampering	Altering data or system components.
R – Repudiation	Denying actions without proof.
I – Information Disclosure	Unauthorised data exposure.
D – Denial of Service	Making services unavailable.
E – Elevation of Privilege	Gaining higher-than-allowed permissions.

◆ DREAD — Risk Rating System

D – Damage Potential	How severe is the impact?
R – Reproducibility	How easily can the attack be repeated?
E – Exploitability	How difficult is it to launch the attack?
A – Affected Users	How many users are impacted?
D – Discoverability	How easy is it to find the weakness?

◆ Other Methodologies (Quick Reference)

PASTA	7-stage, attacker-perspective simulation. Aligns architecture to real attack paths.
Trike	Risk-management oriented. Maps actions, roles, asset permissions. Defines acceptable risk.
VAST	Visual, Agile, Simple. For large enterprises. Integrates into agile workflows.
Attack Tree	Visual hierarchy (AND/OR) of all ways an attacker can reach a goal.
CVSS	Standardised 0–10 severity scoring. Uses impact, exploitability, complexity metrics.
T-MAP	For COTS systems using UML. Calculates risk via asset–threat relationships.

◆ Threat Modelling Tools

Microsoft Threat Modelling Tool, Iriurisk, securiCAD, SD Elements, MyAppSecurity, CVSS 3.0.

C. Access Control

Access control is a security mechanism defining who can access information, systems or physical spaces. It uses Authentication (who are you?) + Authorisation (what can you do?).

▶ Types of Access Control

1. DAC – Discretionary	Owner controls who gets access. Flexible but less secure. Common in personal computers.
2. MAC – Mandatory	Central authority enforces strict security levels. Users can't change permissions. Used in military/government, SELinux.
3. RBAC – Role-Based	Permissions based on job role. HR can't create network accounts. Simplifies management.
4. ABAC – Attribute-Based	Access based on multiple attributes (user, device, environment). Dynamic; used in cloud/zero-trust.
5. Ru-BAC – Rule-Based	Access based on predefined rules/conditions (e.g., time-based). System auto-enforces rules.
6. Or-BAC – Organisation-Based	Policies defined independently of the technical system. Separates policy design from implementation.

▶ Categories of Access Control

Physical Access Control: Controls entry to buildings/rooms. Uses badges, keycards, locks, biometrics.

Logical Access Control: Controls access to networks, systems, data. Uses passwords, MFA, firewalls, permissions.

💡 *PYQ Tip: MAC vs DAC is a frequently tested comparison. In MAC, the admin sets rules; in DAC, the resource owner sets rules.*

📌 **PYQ: 2025 Q1iii – How is MAC different from DAC? | 2025 Q1iv – Authentication techniques for web apps.**

D. Email Security & Attacks

Email security protects messages from unauthorised access, phishing, spam and malware using encryption, spam filters, and 2FA.

Privacy: Only the intended recipient reads the email.

Accuracy: Messages aren't tampered with in transit.

Safety: No viruses or scams pass through.

▶ Types of Email / Phishing Attacks

Phishing	Fraudulent emails mimicking trusted sources to steal credentials or install malware. Uses urgent language, fake links.
Vishing (Voice)	Phone/voicemail attacks. Spoofs caller ID, poses as bank/IT support, asks for OTPs or passwords.
Smishing (SMS)	Phishing via text message. Sends malicious links; visiting installs malware on mobile.
Whaling	Phishing targeting high-profile executives (CEO/CFO). Requests money transfers or sensitive data.

Spear Phishing	Highly targeted phishing using personal info about the victim to seem legitimate.
Pharming	Redirects users to a fake website by altering DNS/browser settings. Victims enter real credentials into fake site.
Angler Phishing	Via social media; impersonates customer service to redirect users to malicious sites.
HTTPS Phishing	Fake sites with HTTPS to appear secure; collects login details.
Clone Phishing	Replicates a real email, replaces links/attachments with malicious ones.
Evil Twin	Fake Wi-Fi hotspot to steal data from users who connect.
Spyware	Software that tracks activity, logs keystrokes, and captures data. Often bundled with legitimate software.

▶ Phishing Attack Flow (4 Steps)

- Attacker sends fraudulent email to victim.
- Victim clicks link and visits phishing website.
- Attacker collects victim's credentials from fake site.
- Attacker uses stolen credentials to access legitimate website.

▶ Protection Against Email Attacks

- Never give personal info in response to unsolicited requests.
- Contact the financial institution directly if suspicious.
- Never share passwords over phone or internet.
- Review account statements regularly.
- Use spam filters, 2FA, and email encryption.

📄 PYQ: 2024 Q3iii – Email-based attacks and countermeasures. | 2025 Q3i – Bait tactics in phishing.

E. OWASP Top 10 (2021)

OWASP = Open Web Application Security Project. Non-profit global community. Releases Top 10 every 3–4 years based on real-world vulnerability data. It is the industry standard security guide for developers.

▶ Quick Overview Table

1. Broken Access Control	Poorly enforced controls → users access others' data. Exploit: IDOR, weak session tokens.
2. Cryptographic Failures	Weak/missing encryption for data at rest or in transit. Use of MD5, SHA1, HTTP.
3. Injection	Malicious code injected via SQL/OS/NoSQL/LDAP. Only apps accepting input are vulnerable.
4. Insecure Design	Flaws in design phase. Can't be fixed by secure implementation. Missing threat modelling.
5. Security Misconfiguration	Default credentials, open admin panels, outdated software. ~95% of cloud breaches are human error.
6. Vulnerable & Outdated Components	Using libraries/frameworks with known CVEs. Attacker targets unpatched systems.

7. Identification & Authentication Failures	Weak auth, no MFA, brute force, default credentials, poor session management.
8. Software & Data Integrity Failures	Untrusted dependencies, insecure CI/CD pipeline, auto-update without signature check.
9. Security Logging & Monitoring Failures	No logging of critical events → attackers operate undetected; incident response hampered.
10. Server-Side Request Forgery (SSRF)	Server fetches user-supplied URL without validation → attacker accesses internal systems.

▶ 1. Broken Access Control — Deep Dive

Occurs when access controls are poorly enforced. Allows authenticated users to reach data/functions they shouldn't.

◆ Vulnerabilities

- No principle of least privilege.
- Insecure Direct Object References (IDOR) — changing URL IDs to access others' data.
- No access controls for PUT/POST/DELETE in APIs.
- Tampering JWT to change role from 'user' to 'admin'.

◆ Example

`http://example.com/object/12345` → change to 12346 to access another user's data (IDOR).

◆ Prevention

- Role-based or attribute-based access control.
- Enforce authorisation server-side.
- Centralise access control logic.
- Apply principle of least privilege.
- Log and monitor access failures.
- Avoid exposing internal IDs directly.

▶ 2. Cryptographic Failures — Deep Dive

Data vulnerable because it is not properly encrypted at rest or in transit.

◆ Vulnerabilities

- Old/weak algorithms (MD5, SHA1) by default.
- Data transmitted without encryption (HTTP, SMTP, FTP).
- Default/weak crypto keys, key reuse, improper key management.
- Server certificate and trust chain not verified.

◆ Prevention

- Use AES-256 for data at rest.
- Use TLS for data in transit.
- Rotate keys regularly; generate using secure random processes.
- Implement MFA for encryption systems.
- Audit crypto systems regularly; never hard-code keys.

▶ 3. Injection — Deep Dive

Attacker inserts untrusted data into an interpreter. Only apps accepting parameters as input are vulnerable.

◆ Causes

- User input not validated/sanitised.
- User input directly concatenated into SQL/command strings.
- ORM queries using raw user input.

◆ Prevention

- Parameterised queries/prepared statements.
- Validate and sanitise all input.
- Least privilege for DB users.
- Deploy Web Application Firewall (WAF).

▶ 4. Insecure Design — Deep Dive

Weaknesses in the design/architecture phase that can't be fixed by secure implementation alone. Missing security requirements during design.

◆ Example

Banking app doesn't require re-authentication for high-value transfers. Attacker on unlocked phone makes unauthorised transfer.

◆ Prevention

- Use Secure Development Lifecycle (SDLC).
- Threat modelling for auth, access controls, and business logic.
- Unit & integration tests for critical flows.
- Document use-cases and misuse-cases.

▶ 5. Security Misconfiguration — Deep Dive

Improper implementation of security settings. Up to 95% of cloud breaches are human errors.

◆ Vulnerabilities

- Default credentials left unchanged.
- Security features disabled in updated systems.
- Outdated software.
- Insecure libraries/databases/frameworks.
- Security headers not set correctly.

◆ Example

Attacker logs into CMS admin panel using default credentials (e.g., admin/admin) and gains full control.

◆ Prevention

- DAST (Dynamic Application Security Testing).
- Disable default passwords.
- Automated verification of security configs.

▶ 6. Vulnerable & Outdated Components — Deep Dive

Open-source libraries/frameworks with known CVEs become weak links in the application.

◆ Example

Apache Struts 2.3 with known RCE vulnerability → attacker exploits, executes arbitrary commands, accesses sensitive data.

◆ Prevention

- Remove unnecessary dependencies.

- Install only from official, secure sources.
- Regularly scan and update all components.

7. Identification & Authentication Failures — Deep Dive

Allows attacker to use manual/automated methods to gain control over accounts. Previously called Broken Authentication.

◆ Causes

- Credential stuffing / brute-force attacks allowed.
- Default/weak passwords permitted.
- No MFA.
- Insecure password recovery.
- Session IDs not properly invalidated.

◆ Prevention

- Implement MFA.
- Protect credentials — never transmit unencrypted.
- Disallow weak passwords.
- Secure credential recovery process.

8. Software & Data Integrity Failures — Deep Dive

Integrity of software/data compromised. Untrusted dependencies or insecure CI/CD pipelines. Auto-update without signature verification.

◆ Example

Auto-update server compromised → pushes malicious update to all clients.

◆ Prevention

- Use trusted repositories only.
- Use digital signatures to verify software.
- Efficient code review process.
- Do not send unencrypted serialised data without integrity check.

9. Security Logging & Monitoring Failures — Deep Dive

Applications fail to log critical events or monitor/alert suspicious activity. Attackers operate undetected; breach response delayed.

◆ Example

E-commerce site suffers brute-force attack. No alerts → accounts compromised before team responds.

◆ Prevention

- Log all login attempts, access control failures, and input validation failures.
- Use a standard log format compatible with SIEM.
- Implement proper incident response plan.
- Effective monitoring and flagging of suspicious activity.

10. Server-Side Request Forgery (SSRF) — Deep Dive

Server fetches user-supplied URLs without validation. Attacker forces server to make requests to internal/unexpected destinations, bypassing firewalls.

◆ Example

Attacker inputs `http://169.254.169.254/latest/meta-data/` (AWS metadata endpoint) → server returns cloud credentials to attacker.

◆ Prevention

- Sanitise and validate all client-supplied URLs.
- Disable HTTP redirections.
- Avoid URL-fetching functionality unless strictly necessary.
- Firewalls/network policies blocking outbound requests to internal systems.

🔗 OWASP Top 10 is the most important section for PYQs from this module.

📌 PYQ: 2023 Q2A – Write short note on OWASP. | 2025 Q5iii – OWASP. | 2023 Q3i – Web application hacking.

F. Secure Code Review

A systematic evaluation of source code to find and remediate security weaknesses before production. Finds vulnerabilities early (logic flaws, injection, auth bugs, secrets).

▶ What is Code Review?

Systematic examination of source code by peers. Performed before merging. Goals: detect bugs, improve readability, verify standards, strengthen security, share knowledge.

Supports building 'self-defending' applications when woven into the SDLC.

▶ Why is Secure Code Review Important?

- Proactive Risk Mitigation — finds openings before expensive breaches.
- Improved Software Quality — more secure and reliable code.
- Compliance Adherence — meets regulatory security standards.
- Enhanced Developer Skills — regular reviews improve team security knowledge.

▶ Elements of Secure Code Review

Static Analysis Tools	Automated scanning for common vulnerabilities and coding errors (e.g., SAST tools).
Manual Review	Experienced devs inspect for logic flaws and design issues that tools miss.
Security Guidelines	Follow OWASP Top 10 and secure coding best practices.
Peer Review	Collaborative review for knowledge sharing; issues identified collectively.
Threat Modelling	Evaluate risks and threats affecting the application.
Vulnerability Remediation	Define appropriate countermeasures for each identified weakness.

G. Quick Comparison & Summary Tables

▶ XSS vs CSRF vs SQLi — Key Differences

Attack	Target / Mechanism
SQLi	Database; injects malicious SQL through input fields.
XSS	Browser; injects malicious JS that executes in victim's browser.

CSRF	Server action; tricks authenticated browser into sending unwanted requests.
-------------	---

▶ Access Control Types — Quick Recap

DAC	Owner decides. Flexible. Used in personal systems.
MAC	Admin decides based on labels/levels. Rigid. Used in military.
RBAC	Job role decides. Most common in enterprises.
ABAC	Multiple attributes decide. Most flexible. Used in cloud.
Rule-Based	Predefined conditions (time, context). Auto-enforced.
Org-Based	Policy independent of technical system.

▶ STRIDE — One-Line Mnemonics

S – Spoofing	I am not who I say I am.
T – Tampering	I changed the data.
R – Repudiation	I didn't do it (but I did).
I – Info Disclosure	You saw what you shouldn't.
D – Denial of Service	You can't use the service.
E – Elevation of Privilege	I got more power than allowed.

H. Previous Year Question Analysis

▶ High-Frequency Topics (Appear in 2+ Papers)

- OWASP Top 10 — 2023, 2025
- Web application hacking / vulnerabilities — 2023, 2024
- Email-based attacks (phishing, vishing, etc.) — 2024, 2025
- Access Control types (MAC vs DAC) — 2025
- Authentication techniques — 2025

▶ 2023 PYQ — Module 3 Relevant Questions

- 📝 PYQ: Q2A: Write short note on OWASP.
- 📝 PYQ: Q3i: 'Website hacked, homepage replaced' — What happened and how to avoid? [Defacement/XSS/Stored XSS]
- 📝 PYQ: Q3ii: Discuss methodologies used for web application hacking.
- 📝 PYQ: Q3iii: Write steps to read email headers and identify SPAM.

▶ 2024 PYQ — Module 3 Relevant Questions

- 📝 PYQ: Q3iii: What are email-based attacks? Explain any one and its countermeasures.
- 📝 PYQ: Q3ii: What is clickjacking? How can a browser detect/block it?

▶ 2025 PYQ — Module 3 Relevant Questions

- 📝 PYQ: Q1iii: MAC vs DAC — How is MAC different?

- 📝 PYQ: Q1iv: Different authentication techniques for web-based applications.
- 📝 PYQ: Q2A alt: What are malwares? Different methods of malware detection.
- 📝 PYQ: Q2Bii: Path traversal attack (../..../etc/passwd) — what went wrong and how to fix.
- 📝 PYQ: Q3i: Bait tactics in phishing attacks.
- 📝 PYQ: Q3ii: Five attacks on browsers.
- 📝 PYQ: Q5iii: Short note on OWASP.

⚡ EXAM TIPS FOR MODULE 3

- OWASP Top 10 is guaranteed — memorise all 10 with one-line descriptions.
- SQLi: always show both the vulnerable code AND the prepared-statement fix.
- XSS: know all 3 types with examples. Reflected = URL, Stored = DB, DOM = JS.
- STRIDE: know the full expansion; it appears in Q5 short-note style.
- Phishing types table: all 11 types listed — learn at least 6.
- MAC vs DAC: a 5-mark comparison is a safe bet every year.
- CSRF: two-line definition + prevention is sufficient for short notes.
- Access Control 6 types — learn the acronyms: DAC, MAC, RBAC, ABAC, Ru-BAC, Or-BAC.

Good luck! 🎯